

PROVA PRÁTICA INDIVIDUAL

React + Vite + TypeScript

Projeto front-end + documentação técnica básica

Código individual

PP-1KAUDEK-1E0EEI6

Aluno

Samuel Melo Santos

Turma

2 DSA

Gerado em

17/09/2026, 07:39

LEITURA OBRIGATÓRIA - CONDIÇÕES DE ZERO

- O projeto deve ser obrigatoriamente desenvolvido em **React + Vite + TypeScript**. Entrega em HTML/JavaScript puro, React com JavaScript, Vue, Angular, Next ou qualquer outra tecnologia no lugar da stack exigida será **zerada**.
- Projeto genérico que não corresponda ao **cenário, código individual, regras e restrições desta folha** será considerado fora do escopo e será **zerado**.
- Leia todos os itens antes de programar. O não cumprimento dos requisitos obrigatórios pode zerar a prova por descaracterização da atividade prática individual.

1. Situação individual

Gestão de Chamados de Manutenção Predial

Uma escola precisa registrar problemas de iluminação, hidráulica, mobiliário e climatização.

Usuário principal: responsável pela manutenção.

Foco sorteado: Histórico e alerta. O foco individual será representar um histórico simples de operações e destacar uma condição de alerta.

Dados mínimos do cenário

- descrição
- categoria
- prioridade
- status

Regras de negócio que devem aparecer no trabalho

- permitir atribuir prioridade
- registrar solução

- registrar local
- destacar chamados urgentes

Regra crítica individual: permitir atribuir prioridade.

Parâmetro individual: use o valor **8** em uma validação, limite, quantidade, capacidade ou alerta coerente com o cenário e explique no PDF onde esse valor foi aplicado.

2. Entregas obrigatórias

Entrega 1 - Requisitos funcionais

1. Escreva **5 requisitos funcionais**, identificados de RF01 a RF05.
 2. Pelo menos **2 requisitos** devem tratar diretamente das regras de negócio acima.
 3. Os requisitos devem descrever comportamento do sistema, não apenas aparência.
-

Entrega 2 - Fluxograma

1. Crie **1 fluxograma** do processo de **atendimento**.
2. O fluxograma deve possuir: início, entrada de dados, pelo menos uma decisão, caminho de erro/negação, processamento e fim.
3. A decisão central deve representar esta regra: **permitir atribuir prioridade**.
4. Entregue o fluxograma em imagem e também dentro do PDF final.

Entrega 3 - Projeto React + Vite + TypeScript

Implementação simplificada: não é necessário backend, API, autenticação real nem integração do React com o banco de dados. Os dados da interface podem ficar em memória usando arrays e `useState` .

1. Crie o projeto com template **React + TypeScript** do Vite.
2. A aplicação deve exibir no cabeçalho o código `PP-1KAUDEK-1E0EEI6` e o nome do sistema **Gestão de Chamados de Manutenção Predial**.
3. Use no mínimo **3 componentes próprios**, sendo pelo menos um componente filho recebendo dados por **props**.
4. Use **useState** para controlar dados e interação.
5. Defina pelo menos **1 interface ou type** do TypeScript para os dados principais.
6. Inclua formulário, listagem e pelo menos uma interação coerente com o foco sorteado.
7. Comece com pelo menos **5 registros simulados** relacionados ao cenário.
8. A aplicação deve registrar eventos em uma lista de histórico e exibir um alerta visual quando a condição crítica do cenário ocorrer.
9. **Restrição técnica individual:** Quando o filtro não encontrar resultado, mostre uma mensagem de lista vazia em vez de deixar a área em branco.
10. A interface deve ser utilizável em tela de notebook e celular, sem conteúdo principal cortado horizontalmente.

Entrega 4 - Banco de dados básico

Importante: o banco de dados é uma entrega de modelagem e SQL básico. **Não precisa ser conectado ao projeto React.**

1. Crie um **DER simples com 3 entidades**: uma para **chamado**, uma relacionada a **local** e uma para registrar **atendimento**.
2. Indique chaves primárias, pelo menos uma chave estrangeira e as cardinalidades.
3. Crie um arquivo `banco.sql` com **CREATE TABLE** para as 3 tabelas e **3 INSERTs em cada tabela**.
4. Use tipos de dados simples e coerentes. Não é necessário criar procedure, trigger, view ou autenticação de banco.
5. **Restrição individual do banco:** Represente no DER um relacionamento 1:N entre o cadastro principal e a operação.

Entrega 5 - Casos de teste

1. Documente **3 casos de teste manuais** em tabela com: ID, requisito relacionado, pré-condição, passos e resultado esperado.
2. Os três testes devem cobrir um caso válido, um caso com campo obrigatório vazio e um caso que viole a regra crítica do cenário.
3. Pelo menos um teste deve verificar a regra crítica individual.

Entrega 6 - Infraestrutura e ambiente

No PDF, informe as versões efetivamente usadas de: sistema operacional, Node.js, npm, Vite, React, TypeScript, navegador e SGBD escolhido para o script.

Entrega 7 - Versionamento obrigatório no Git

1. O projeto deve estar em **repositório público** e acessível pelo link enviado no Google Classroom.
2. O README deve começar com o nome do aluno, nome do sistema e código `PP-1KAUDEK-1E0EEI6`.
3. Faça no mínimo **4 commits significativos** durante a produção. Sugestão de sequência: `início: PP-1KAUDEK-1E0EEI6`, `interface: PP-1KAUDEK-1E0EEI6`, `logica: PP-1KAUDEK-1E0EEI6` e `final: PP-1KAUDEK-1E0EEI6`.
4. **Regra de integridade:** depois da data/hora oficial de encerramento da prova informada pelo professor, **não altere o repositório**. Havendo commit/atualização posterior ao prazo oficial, **a parte de versionamento será zerada**.
5. O repositório deve conter o código-fonte necessário para executar o projeto. Não envie apenas screenshots ou arquivos compilados.

3. Evidências contra entrega genérica

Como existe a possibilidade de uso de ferramentas de IA, a avaliação considera principalmente a correspondência entre o escopo individual, o código-fonte, o histórico de desenvolvimento e a capacidade de explicar o que foi construído. Por isso, inclua no PDF:

- **1 screenshot da aplicação executando**, mostrando o código individual no cabeçalho.
- **1 screenshot do histórico de commits** do repositório.
- **Uma justificativa de 5 a 8 linhas** explicando como a regra crítica *“permitir atribuir prioridade”* foi tratada no código.
- **Uma explicação curta** indicando onde props, useState e o type/interface do TypeScript foram usados.

O professor poderá solicitar que o aluno explique trechos do próprio projeto e demonstre uma pequena alteração coerente com o código entregue.

4. Entrega pelo Google Classroom

COMPROVAÇÃO OBRIGATÓRIA DO ESCOPO: após gerar esta prova, use o botão **“Imprimir / Salvar em PDF”** e salve o **PDF da prova individual gerada**. Esse arquivo deve ser enviado no Google Classroom **sem alterar o texto da prova**, pois será utilizado para comprovar exatamente qual situação, código, regras e requisitos foram atribuídos ao aluno.

- ☐ **PDF da prova individual gerada por este sistema**, salvo pelo botão “Imprimir / Salvar em PDF”, para comprovação do escopo recebido.
- ☐ PDF final do trabalho com requisitos, fluxograma, DER, testes, infraestrutura e evidências.
- ☐ Arquivo banco.sql .
- ☐ Imagens do fluxograma e DER, caso não estejam separadas no PDF.
- ☐ Link do repositório público do Git.

Importante: existem dois PDFs diferentes na entrega: (1) o PDF desta prova individual gerada, usado como comprovante do escopo; e (2) o PDF final produzido pelo aluno com a documentação e as evidências do projeto. O código-fonte principal ficará no repositório. Se o professor solicitar arquivo compactado adicional, ele deve corresponder ao mesmo estado final publicado no Git.

5. Checklist final antes de enviar

- ☐ Projeto é React + Vite + TypeScript.
- ☐ Código individual aparece na aplicação e no README.
- ☐ 5 requisitos funcionais.
- ☐ Fluxograma com decisão e caminho de erro.
- ☐ 3+ componentes, props, useState e type/interface.
- ☐ Restrição técnica e parâmetro individual implementados.
- ☐ DER com 3 entidades e cardinalidades.
- ☐ banco.sql com tabelas e dados básicos.

- ☐ 3 casos de teste documentados.
- ☐ Versões do ambiente registradas.
- ☐ Repositório público com 4+ commits.
- ☐ PDF da prova individual gerada salvo e anexado para comprovação.
- ☐ Todos os documentos e link enviados no Classroom.

Prova individual: PP-1KAUDEK-1E0EEI6 - Samuel Melo Santos